

LAB 08

Title: Computing FIRST Set of a Context-Free Grammar Using C++ with Debugging Techniques.

Objective

To learn how to debug a C++ program using Breakpoints and the Watch Window, and to understand how the FIRST set of each non-terminal in a context-free grammar is computed using recursion.

Procedure

1. Complete environment setup.
2. Write the FIRST set computation code.
3. Set break points.

Breakpoint Location	Reason
if(!isupper(c))	To check whether the current symbol is a terminal, which forms the base case of the recursion.
first[n][firstCount[n]++] = c;	To verify that a terminal symbol is directly added to the FIRST set.
for(int i = 0; i < 10; i++)	To observe each production rule being scanned for the given non-terminal.

4. Execute with Next Line.
5. Track the changes on Watch.

Code with break-points

```
[*] Pratices.cpp  ✕
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  char leftSide[10];
5  string rightSide[10];
6
7  char first[10][10];
8  int firstCount[10];
9
10 void findFirst(int n, char c)
11 {
12     // Terminal
13     if(!isupper(c))
14     {
15         first[n][firstCount[n]++] = c;
16         return;
17     }
18
19     for(int i = 0; i < 10; i++)
20     {
21         if(leftSide[i] == c)
22         {
23             string s = rightSide[i];
24
25             // Epsilon
26             if(s == "#")
```

```

27     {
28         first[n][firstCount[n]++] = '#';
29         continue;
30     }
31
32     char x = s[0];
33
34     // Terminal
35     if(!isupper(x))
36     {
37         first[n][firstCount[n]++] = x;
38     }
39     else
40     {
41         findFirst(n, x);
42     }
43     }
44 }
45
46
47 int main()
48 {
49     int n;
50     cin >> n;

```

```

52     // Input
53     for(int i = 0; i < n; i++)
54     {
55         string s;
56         cin >> s;
57
58         leftSide[i] = s[0];
59         rightSide[i] = s.substr(3);
60     }
61
62     // FIRST
63     for(int i = 0; i < n; i++)
64     {
65         findFirst(i, leftSide[i]);
66     }
67
68     // Output
69     for(int i = 0; i < n; i++)
70     {
71         cout << "FIRST(" << leftSide[i] << ") = { ";
72
73         for(int j = 0; j < firstCount[i]; j++)
74         {
75             cout << first[i][j] << " ";
76         }

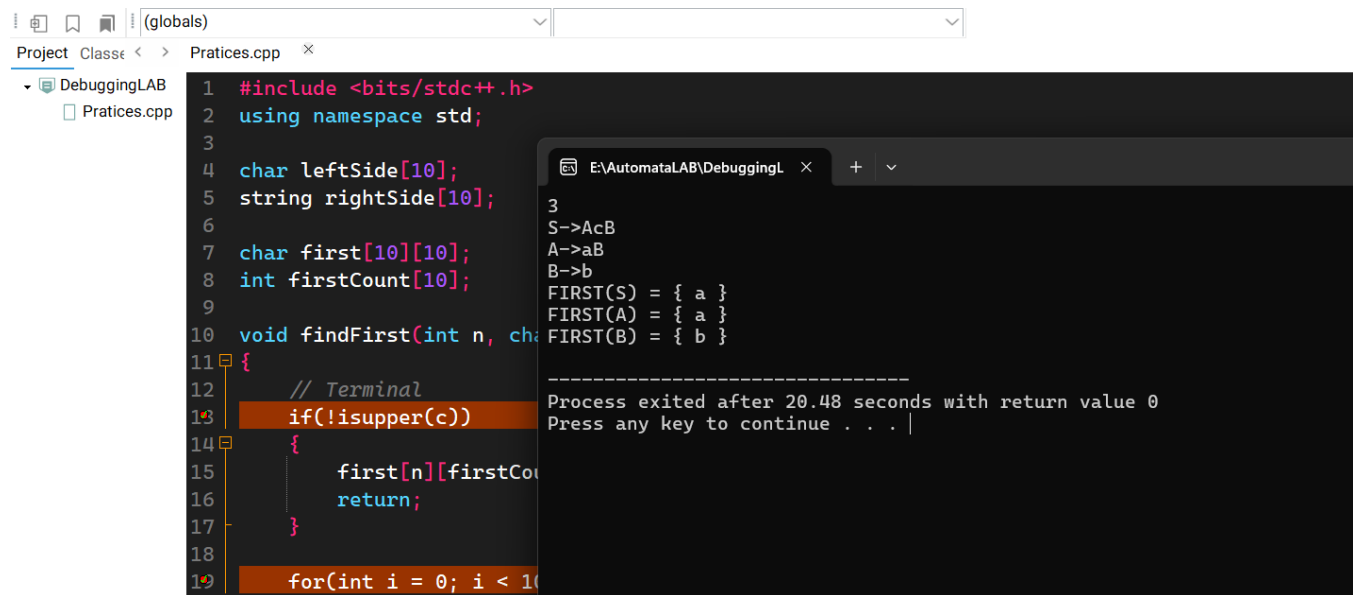
```

```

76     }
77
78     cout << "}\n";
79 }
80
81 return 0;
82 }

```

Output



The screenshot shows a C++ IDE with a source code editor on the left and a debugger window on the right. The source code is for a program that finds the first occurrence of a character in a string. The debugger window shows the execution state, including the current line of code and the values of variables.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 char leftSide[10];
5 string rightSide[10];
6
7 char first[10][10];
8 int firstCount[10];
9
10 void findFirst(int n, char c)
11 {
12     // Terminal
13     if(!isupper(c))
14     {
15         first[n][firstCount[n]] = c;
16         firstCount[n]++;
17     }
18 }
19
20 for(int i = 0; i < 10; i++)
```

The debugger window shows the following output:

```
3
S->AcB
A->aB
B->b
FIRST(S) = { a }
FIRST(A) = { a }
FIRST(B) = { b }
-----
Process exited after 20.48 seconds with return value 0
Press any key to continue . . . |
```